

Software for Bayesian

UBCO MDS — DATA 582





- ▶ We've covered the basics of Bayesian inference over the last few weeks, today we'll introduce software and show a bunch of examples.
- ▶ There are several options for broadly applicable Bayesian analysis software. Let's start with a quick overview of the options...

► OpenBUGS

- Pros - simple syntax (similar to R), can be called from R through some packages
- Cons - older backend framework focused on Windows environment, Gibbs sampling focus can be slow in high parameter dimensions, especially with correlated parameters

► JAGS

- Pros - easier setup in any OS, basically a slightly more modern version of BUGS, rjags can call to it
- Cons - still slow in high parameter dimensions, especially with correlated parameters



- ▶ Stan
 - ▶ Pros - more efficient sampling scheme (HMC, NUTS), rstan and PyStan, can be used for complicated models
 - ▶ Cons - some mildly specialized syntax

- ▶ PyMC
 - ▶ Pros - based on same sampling approach to Stan, GPU acceleration faster for large datasets, natural syntax in Python
 - ▶ Cons - can be slower than Stan on small-to-medium data sets, Python-only

- ▶ Since Stan is more computing-environment agnostic (and will allow me to use R), let's focus our efforts there.
- ▶ Aside: Stan is named after Stanislaw Ulam, a mathematician who devised the Monte Carlo method¹
- ▶ More Aside: Stan Ulam published this method from Los Alamos Laboratory, while also working out how to initiate fusion in the hydrogen bomb.

¹Metropolis and Ulam (1949). 'The Monte Carlo method'. *Journal of the American Statistical Society*

- ▶ Let's conduct MCMC simulation using the [rstan](#) package; R's interface to [Stan](#).
- ▶ There are two essential steps to all rstan analyses:
 - ▶ Step 1. define the Bayesian model structure in rstan notation
 - ▶ Step 2. simulate the posterior.
- ▶ Stan models are written in their own *syntax*.
- ▶ We will look at a few familiar models to gain some familiarity with the key elements of an rstan simulation.

2

²Examples and explanations were adapted from: the [Stan Reference Manual](#), [BayesRules](#), and [RStan YouTube channel](#)

- ▶ Stan programs are made up of a series of **blocks**
- ▶ The blocks are made up of variable **declarations** and **statements**
- ▶ Variables must be defined to have some value (as well as declared³ to have some type) before they are used — if they do not, the behavior is undefined.
- ▶ Stan allows local variables to be declared in blocks

³This follows the convention of programming languages like C++ and Fortran, not the conventions of scripting languages like Python or R.

- ▶ We will be looking at three types of blocks in today's examples

`data` The **data block** declares the required data for the model.

`parameters` The **parameters block** declares the model parameters

`model` The **model block** is where we specify our prior(s) and likelihood

- ▶ The models we will look at will all have a similar skeleton; however, there are other types of blocks available; see [here](#) for an overview.

- ▶ Variable in Stan must have a declared data type. For example,
 - ▶ `real` for continuous values and `int` for integers,
 - ▶ `vector` for column vectors, `row_vector` for row vectors, and `matrix` for matrices.

An overview of data types [here](#).

- ▶ This forces the user to be intentional and also serves as a way of catching value errors as soon as they occur.
- ▶ In the `data` block and `parameters` blocks are solely used to declare the data variables for input and the parameter variables for sampling.

Declarations

Constrained Data Types



- ▶ Declarations of variables may be provided with constraints.
- ▶ All of the basic data types⁴ may be given lower and/or upper bounds using the following syntax:

```
int a = 5; \\ this defines a static local variable  
int<lower=1> N;  
real<upper=0> log_p;  
vector<lower=-1, upper=1>[3] rho; \\ vector of length 3
```

⁴other than complex

- ▶ Each variable's value is validated against its declaration as it is read. For example,
 - ▶ if you try to assign a negative value to a variable declared `real<lower=0>`, an error will be raised,
 - ▶ if you provide data of the wrong size to a vector, an error will be raised

- ▶ As a result, data type errors will be caught as early as possible.

- ▶ The variables declared in the `parameters` block correspond directly to the variables being sampled by Stan's samplers.
 - ▶ These samplers with HMC (Hamiltonian Monte Carlo) and NUTS⁵ (no U-turn sampler).
- ▶ The `model` block consists of specifying the likelihood and the priors.
- ▶ They typically consist of defining a probability model using a **sampling statement**.

⁵Hoffman, Matthew D., and Andrew Gelman. 2014. "The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo." *Journal of Machine Learning Research* 15: 1593–623. [\[1\]](#)

- ▶ In general, a **sampling statement** has the form⁶

```
y ~ dist(theta1, ..., thetaN);
```

- ▶ For instance, we might have a statement like:

```
y ~ binomial(10, theta);
```

or

```
y ~ normal(mu, sigma);
```

⁶model blocks allow sampling statements while data and parameter blocks do not.



- ▶ All of the variables declared in the data block must be read into memory. If a variable cannot be read, the program will halt with a message indicating which data variable is missing.
- ▶ Stan can read this from a file or read it directly from memory in R.
- ▶ If the variable has a declared constraint, it will be validated.
- ▶ If the constraint is violated, the program will halt with a warning message indicating which variable contains an illegal value, the value that was read, and the constraint that was declared.



- ▶ Consider the following Beta-Binomial model for our MLB rookie (again, game one 3 hits during 5 at bats)

$$Y|p \sim \text{Binomial}(5, p)$$
$$p \sim \text{Beta}(30, 100).$$

Step 1

Defining the Bayesian model structure



We must specify three aspects upon which it depends:

- ▶ `data` y the observed number of successes in 5 trials.
 - ▶ we know this must be an integer between 0 and 5.
- ▶ `parameters` the model depends on one parameter p
 - ▶ we know this must be a real number between 0 and 1
- ▶ `model` Here we identify the prior on our parameter p and the form our likelihood takes on.
 - ▶ $Y \mid p \sim \text{Binomial}(5, p)$; specified using `binomial()`
 - ▶ $p \sim \text{Beta}(30, 100)$; specified using `beta()`

Step 1

data block



Our data, y , is the observed number of successes in 5 trials. We know this must be an integer between 0 and 5.

So our data block is given by:

```
data {  
  int<lower = 0, upper = 5> y;  
}
```

Here we declare any of the data that we are going to pass to stan. The input data is usually stored in list. These declarations tell Stan what this data list should look like.

Step 1

parameters block



Our model depends on one parameter p which we know this must be a real number between 0 and 1

So our parameters block is given by:

```
parameters {  
  real<lower = 0, upper = 1> p;  
}
```

Here we declare what are the parameters that we wish to infer.

Step 1

model block



Our prior and likelihood take the form:

- ▶ $Y \mid p \sim \text{Binomial}(5, p)$; specified using `binomial()`
- ▶ $p \sim \text{Beta}(30, 100)$; specified using `beta()`

So our model block is given by:

```
model {  
  y ~ binomial(5, p); // likelihood  
  p ~ beta(30, 100);   // prior  
}
```

Here we declare the prior and the likelihood

⁷

⁷the use of two backslashes to denote the start of a comment

- ▶ We translate this structure into [rstan syntax](#) and store it as the *character string*.
- ▶ We obviously won't be able to go through all the details on the syntax in the time left if this module, but as you will see, they are pretty intuitive.
- ▶ If you are interest in more, see [Stat documentation](#) for user guides and reference manuals
- ▶ Although it can also be done using a character string within R (as we will demo in this example) Rstan [recommends](#) using a separate file with a `.stan` extension. In RStudio:

New File → Stan file

```
# STEP 1: DEFINE the model
# alternatively we can save this to *some_name.stan*
bball_model <- "
  data {
    int<lower = 0, upper = 5> y;
  }

  parameters {
    real<lower = 0, upper = 1> p;
  }

  model {
    y ~ binomial(5, p);    // likelihood
    p ~ beta(30, 100);    // prior
  }
"
```

Step 2



- ▶ In **step 2**, we simulate the posterior
- ▶ We do this using the `stan` function from the **rstan** library.
- ▶ Very loosely speaking, `stan()` designs and runs an MCMC algorithm to produce an approximate sample from the Beta-Binomial posterior.

```
# STEP 2: SIMULATE the posterior
bball_sim <- stan(model_code = bball_model, data = list(y = 3
  chains = 4, iter = 5000*2, seed = 84735)
```

Required arguments

`model_code` the object name for the stan model (here `bball_model`)
`data` a *list* of the observed data (here `y = 3`).

Optional arguments

`chains` how many parallel Markov chains to run (default is 4 chains)
`iter` length of each Markov chain⁸
`seed` random number generating seed (good practice for reproducibility).

⁸By default, the first half of these iterations are thrown out as burn-in

- ▶ The result, stored in `bb_sim`, is a `stanfit` object.
- ▶ This object includes four parallel Markov chains run for 10,000 iterations each.
- ▶ After tossing out the first 5,000 iterations of all four chains, we end up with four separate Markov chain samples of size 5,000, or a combined Markov chain sample size of 20,000.

- ▶ `print` provides a summary for the parameter of the model
- ▶ For more detailed statistics you can alternatively use `summary`.⁹
- ▶ The R package `shinystan` can provide several diagnostic and inferential plots...

⁹For more methods and details of class `stanfit`, see the help of class `stanfit`, i.e `?stanfit`

```
> print(bball_sim)
Inference for Stan model: anon_model.
4 chains, each with iter=10000; warmup=5000; thin=1;
post-warmup draws per chain=5000, total post-warmup draws=20000.
```

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
p	0.24	0.00	0.04	0.17	0.22	0.24	0.27	0.32	8109	1
lp__	-75.59	0.01	0.72	-77.62	-75.76	-75.31	-75.13	-75.08	6923	1

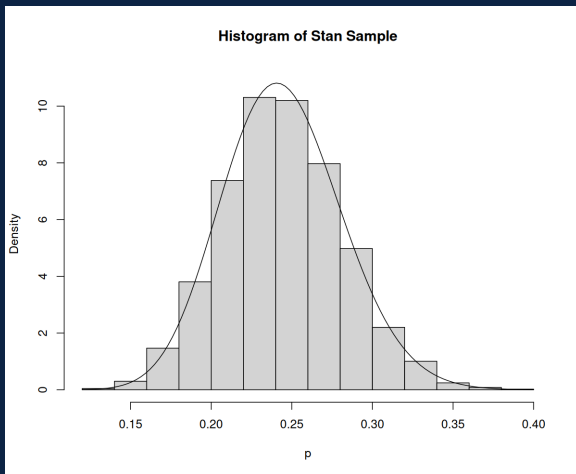
Samples were drawn using NUTS(diag_e) at Sun Apr 19 22:05:58 2026.
For each parameter, `n_eff` is a crude measure of effective sample size,
and `Rhat` is the potential scale reduction factor on split chains (at
convergence, `Rhat=1`).

► Reminder, our 95% Credible Interval for this example was (0.176,0.320)!

True Posterior vs Stan Posterior Sample



- Curve generated by a bunch of math, histogram of samples generated by a bunch of compute!



Linear Regression Example

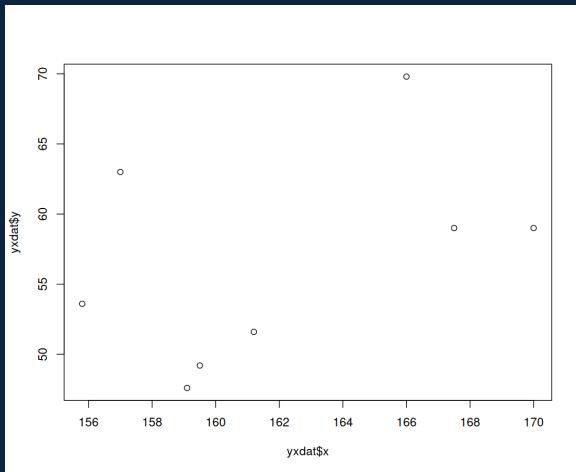


- ▶ Let's take on a 'hypothetical' SLR example with real data
- ▶ Historically, medical studies have had (white) men overrepresented as subjects. This has led to countless misdiagnoses, etc.
- ▶ But, suppose for this reason, we had a very good understanding surrounding the linear model for weight (response, in kg) given height (predictor, in cm) for men.
- ▶ We decide to take advantage of this knowledge, under the assumption that the relationship between these variables is approximately the same across reported gender.
- ▶ We assume
 - ▶ $Y \sim N(\beta_0 + \beta_1 X, \sigma^2)$
 - ▶ $\beta_0 \sim N(-61, 15^2)$
 - ▶ $\beta_1 \sim N(0.8, 0.1^2)$
 - ▶ $\sigma^2 \sim \text{Inv-Gam}(3, 160)$

Linear Regression Example



- For whatever reason, we could only initially collect 8 samples (height and weight) from subjects who self-reported as female.



Linear Regression Example: Frequentist



► The frequentist approach gives...

```
> smlm <- lm(y~x, data=yxdat)
> summary(smlm)
```

Call:

```
lm(formula = y ~ x, data = yxdat)
```

Residuals:

Min	1Q	Median	3Q	Max
-7.144	-4.812	-1.892	3.117	10.660

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-46.6141	86.0743	-0.542	0.608
x	0.6371	0.5310	1.200	0.275

Residual standard error: 7.3 on 6 degrees of freedom

Multiple R-squared: 0.1935, Adjusted R-squared: 0.05904

F-statistic: 1.439 on 1 and 6 DF, p-value: 0.2755



- ▶ We decide to take advantage of our prior knowledge regarding men, under the assumption that the relationship between these variables is approximately the same across reported gender.
- ▶ We assume
 - ▶ $Y \sim N(\beta_0 + \beta_1 X, \sigma^2)$
 - ▶ $\beta_0 \sim N(-61, 15^2)$
 - ▶ $\beta_1 \sim N(0.8, 0.1^2)$
 - ▶ $\sigma^2 \sim \text{Inv-Gam}(3, 160)$

Linear Regression Example: Stan



```
# STEP 1: DEFINE the model
slr_model <- "
  data {
    vector[8] y;
    vector[8] x;
  }

  parameters {
    real beta0;
    real beta1;
    real<lower=0> sigma2;
  }

  model {
    y ~ normal(beta0 + beta1*x, sqrt(sigma2)); // likelihood
    beta0 ~ normal(-61, 15); // prior intercept
    beta1 ~ normal(0.8, 0.1); // prior slope
    sigma2 ~ inv_gamma(3, 160); // prior error variance
  }
"
```


Linear Regression Example: Stan



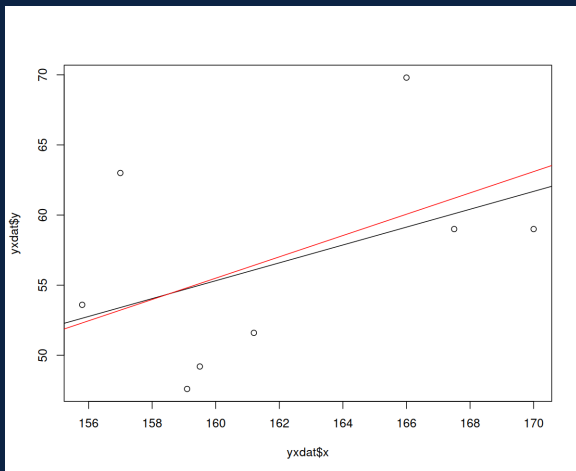
```
# STEP 2: SIMULATE the posterior
slr_sim <- stan(model_code = slr_model, data = yxdat,
               chains = 4, iter = 5000*2, seed = 84735)
print(slr_sim)
Inference for Stan model: anon_model.
4 chains, each with iter=10000; warmup=5000; thin=1;
post-warmup draws per chain=5000, total post-warmup draws=20000.
```

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
beta0	-66.10	0.14	11.00	-87.64	-73.43	-66.14	-58.75	-44.58	6380	1
beta1	0.76	0.00	0.07	0.62	0.71	0.76	0.80	0.89	6338	1
sigma2	58.35	0.32	27.27	26.06	40.27	51.79	69.05	127.54	7182	1
lp__	-35.51	0.02	1.29	-38.82	-36.10	-35.18	-34.56	-34.04	6135	1

Linear Regression Example



- Bayesian point estimate line (red) and frequentist OLS (black)





- ▶ For better or worse, we are effectively borrowing the knowledge from previous studies on men.
- ▶ This assumption is allowing us to make much stronger inferences than the frequentist approach allows.
- ▶ As our sample size grows, the data will be less reliant on the prior distributions (in this case, our prior beliefs based on male subjects).

Linear Regression Example: Stan



```
# STEP 1: DEFINE the model
slr_model <- "
  data {
    vector[260] y;
    vector[260] x;
  }

  parameters {
    real beta0;
    real beta1;
    real<lower=0> sigma2;
  }

  model {
    y ~ normal(beta0 + beta1*x, sqrt(sigma2)); // likelihood
    beta0 ~ normal(-61, 15); // prior intercept
    beta1 ~ normal(0.8, 0.1); // prior slope
    sigma2 ~ inv_gamma(3, 160); // prior error variance
  }
"
```

Linear Regression Example: Stan



```
# STEP 2: SIMULATE the posterior
slr_sim_full <- stan(model_code = slr_model_full, data = yxdatfull,
                    chains = 4, iter = 5000*2, seed = 84735)

print(slr_sim_full)

Inference for Stan model: anon_model.
4 chains, each with iter=10000; warmup=5000; thin=1;
post-warmup draws per chain=5000, total post-warmup draws=20000.
```

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff
beta0	-56.88	0.11	8.61	-73.58	-62.70	-56.89	-51.04	-40.04	5662
beta1	0.71	0.00	0.05	0.61	0.68	0.71	0.75	0.81	5657
sigma2	75.69	0.08	6.65	63.66	71.05	75.36	79.96	89.63	7142
lp__	-708.74	0.02	1.22	-711.88	-709.29	-708.42	-707.84	-707.34	5593

Linear Regression Example: Frequentist



► The frequentist approach gives...

```
> flm <- lm(y~x, data=yxdatfull)
> summary(flm)
```

Call:

```
lm(formula = y ~ x, data = yxdatfull)
```

Residuals:

Min	1Q	Median	3Q	Max
-14.315	-6.099	-1.426	3.844	43.355

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-43.81929	13.61878	-3.218	0.00146 **
x	0.63334	0.08254	7.673	3.45e-13 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 8.693 on 258 degrees of freedom

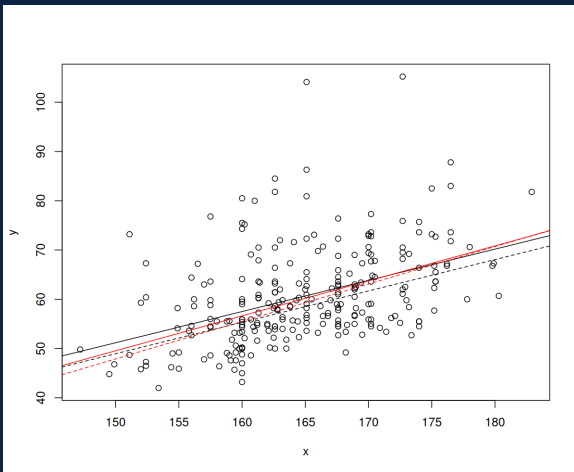
Multiple R-squared: 0.1858, Adjusted R-squared: 0.1827

F-statistic: 58.88 on 1 and 258 DF, p-value: 3.448e-13

Linear Regression Example



- Bayesian point estimate line (red, dashed $n=8$) and frequentist OLS (black, dashed $n=8$)



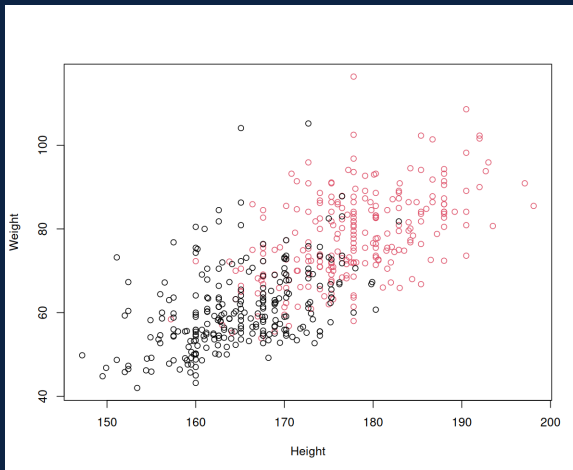


- ▶ The largest contrast can consider the Bayesian Crl versus Frequentist CI
- ▶ For $n=8$, looking at β_1
 - ▶ Freqs 95% CI: $\approx (-0.42, 1.7)$
 - ▶ Bayes 95% Crl: $\approx (0.62, 0.89)$
- ▶ For $n=260$, looking at β_1
 - ▶ Freqs 95% CI: $\approx (0.47, 0.79)$
 - ▶ Bayes 95% Crl: $\approx (0.61, 0.81)$

Linear Regression Example



- FYI, data that I based this on (reported male in red)



- ▶ Time permitting, jump to shinytan!



THE UNIVERSITY OF BRITISH COLUMBIA

